Learning Word Embeddings from Scratch

Skip-gram with Negative Sampling on the text8 Corpus

Natural Language Processing — Project Presentation

Nguyễn Bá Anh 20261218M
Nguyễn Thị Nhã Linh 20261262M

Instructor: Nguyễn Kiêm Hiếu

Motivation: words need numbers

One-hot vectors

$$king = [0, \ldots, 0, 1, 0, \ldots, 0] \in \mathbb{R}^V$$

- as long as the vocabulary ($V \approx$ 36,280 here)
- every pair of words is orthogonal: *king* is as far from *queen* as from *carrot*

Word embeddings

$$king \mapsto \mathbf{v}_{king} \in \mathbb{R}^{100}$$

- dense, learned from text
- words used in similar contexts get similar vectors

"You shall know a word by the company it keeps." — Firth (1957)

Question

How can a model learn useful numerical representations of words from their surrounding context?

What is Word2Vec?

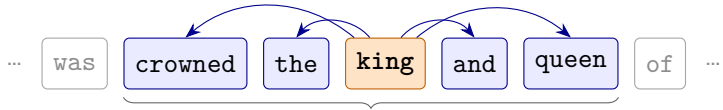
- A shallow model with two embedding matrices (Mikolov et al., 2013a)
  - W_{in}: one vector $\mathbf{v}_w$ per word as a **centre** word ← the embeddings we keep
  - W_{out}: one vector $\mathbf{v}'_w$ per word as a **context** word
- Two architectures: **CBOW** predicts the centre word from its context; **Skip-gram** predicts each context word from the centre word

Skip-gram with a full softmax:

$$p(o \mid c) = \frac{\exp(\mathbf{v}'_o \cdot \mathbf{v}_c)}{\sum_{w=1}^{V} \exp(\mathbf{v}'_w \cdot \mathbf{v}_c)}$$

The denominator sums over all V words, for every one of the 60.7M training pairs.

Skip-gram: training pairs from a context window



Positive pairs (label 1)

`(king, crowned)`
`(king, the)`
`(king, and)`
`(king, queen)`

Negative pairs (label 0)

`(king, river)`, `(king, seven)`, …

K random words per positive pair, drawn from $P_n(w) \propto f(w)^{3/4}$

- Window size drawn at random, $W \sim U\{1, \ldots, 5\}$: nearby words count more
- About 6 positive pairs per centre word

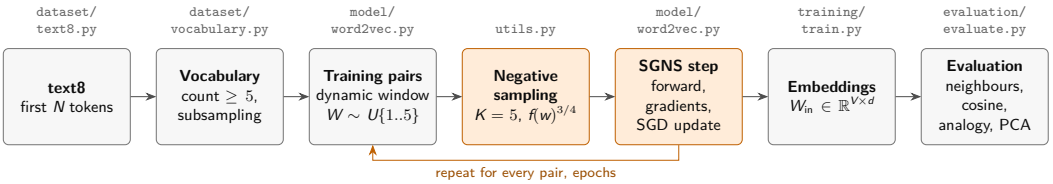
Negative sampling: a binary task instead of a softmax

Mikolov et al. (2013b): tell the true pair apart from K random pairs

$$\underbrace{\log \sigma(\mathbf{v}'_o \cdot \mathbf{v}_c)}_{\text{pull the true context closer}} + \underbrace{\sum_{k=1}^{K} \log \sigma(-\mathbf{v}'_{n_k} \cdot \mathbf{v}_c)}_{\text{push random words away}}, \qquad n_k \sim P_n(w) \propto f(w)^{3/4}$$

- Cost per pair: $O(K)$ with $K = 5$, instead of $O(V)$
- The exponent $3/4$ samples rare words more often than their raw frequency
- **Subsampling**: drop occurrences of very frequent words (*the*, *of*) with probability $1 - \sqrt{t/f(w)}$

Implementation



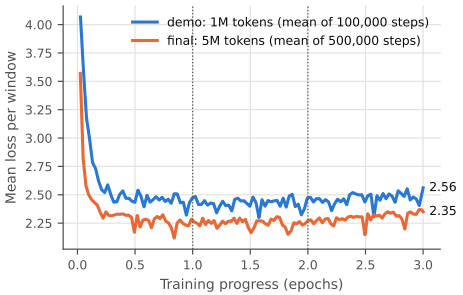
NumPy SGNS from the open-source repository of Caselli (2026). Our additions:

Reproducible runs
seed, every hyperparameter on the CLI, `config.json` per run; fixed the documented command

4× faster training
negatives drawn in batches from the same distribution: 1,023 s → 251 s on 1M tokens, same losses

Evaluation protocol
queries fixed in advance, untrained baseline, frequency-matched cosine percentiles

Results: training loss and nearest neighbours



- Falls early in epoch 1, then flat
- Epoch means 2.347 → 2.250 → 2.299
  ⇒ loss is **not** a quality measure

Query	Nearest neighbours (top 4)
king	alexius, grandson, throne, vii
computer	computers, computing, interface, hardware
city	town, edmonton, bronx, bremerhaven
france	italy, spain, netherlands, switzerland
january	february, december, october, june
three	seven, four, six, two
war	peloponnesian, ww, wwii, outbreak
music	jazz, musical, dance, pop

Final run (5M tokens); queries fixed in advance.
Untrained model: *king* → *marpat, verbally, brando*

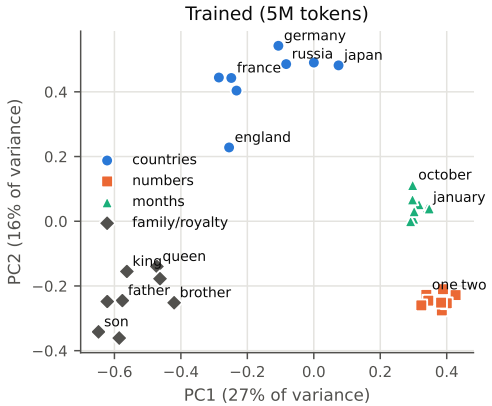
Live demo: similarity, analogies, visualisation

```
$ python demo/demo.py

> king queen
  0.699: more similar than 99.7%
         of random frequent-word pairs
> king software
  0.123: more similar than 5.0%

> bigger - big + small
  smaller  parking  forested
> king - man + woman
  daughter  caligula  amalric
```

Analogies: **4/8** exact, 6/8 in top 5
(1M tokens: 0 and 2; untrained: 0)



Conclusion

Q1 Loss	drops early, then flat; says little about embedding quality
Q2 Neighbours	**yes**: months, numbers, countries, genres cluster
Q3 Similarity	**yes**, against the right baseline: related pairs $\geq$ p85, controls $\leq$ p32
Q4 Analogies	**partly**: 4/8 exact; *king* $-$ *man* $+$ *woman* $\not\to$ *queen*
Q5 Space	**yes**: four word groups form separate clusters in 2D

Limitations

- 5M tokens, a prefix of Wikipedia
- rare words barely trained
- one vector per word (*bank*)
- small fixed evaluation, one seed

Applications

- input features for text classifiers
- similarity search, query expansion
- exploring how a corpus uses words

Backup slides

Backup: runs and hyperparameters

	Demo	Final
Tokens of text8	1,000,000	5,000,000
Vocabulary size	13,966	36,280
Tokens after subsampling	635,140	3,372,979
Training steps (pairs)	11,425,544	60,708,682
Training time	4.2 min	24.7 min
Throughput (steps/s)	45,500	40,900
Mean loss, epoch 1	2.591	2.347
Mean loss, epoch 2	2.424	2.250
Mean loss, epoch 3	2.468	2.299
Random-pair cosine (top 2,000)	0.61 ± 0.19	0.32 ± 0.12
Analogies: hit@1 / hit@5	0 / 2 of 8	4 / 6 of 8

Hyperparameter	Value	CLI option
Tokens of text8 used	1,000,000 (demo) / 5,000,000 (final)	`--max-tokens`
Minimum word count	5	`--min-count`
Subsampling threshold t	0.001	`--subsample-t`
Embedding dimension d	100	`--embed-dim`
Epochs	3	`--epochs`
Maximum window size	5	`--max-window`
Negative samples K	5	`--n-negatives`
Noise distribution exponent	0.75	fixed
Initial / minimum learning rate	0.025 / 0.0001	`--start-lr`, `--lr-min`
Loss report interval (steps)	100,000 / 500,000	`--report-every`
Random seed	42	`--seed`

Apple M5, one core, Python 3.11.17, NumPy 2.4.6. Same arguments $\Rightarrow$ identical embeddings.

Backup: cosine similarity needs a baseline

Pair	Demo (1M)	Final (5M)	Untrained
king – queen	0.91 (p98)	0.70 (p100)	0.10 (p82)
man – woman	0.89 (p97)	0.77 (p100)	−0.06 (p30)
computer – software	0.80 (p84)	0.75 (p100)	0.17 (p96)
city – country	0.78 (p81)	0.45 (p85)	−0.14 (p9)
france – germany	0.86 (p93)	0.76 (p100)	0.05 (p68)
january – february	0.95 (p100)	0.91 (p100)	−0.03 (p39)
king – software	0.14 (p1)	0.12 (p5)	0.00 (p49)
computer – january	0.42 (p17)	0.18 (p12)	0.04 (p67)
music – france	0.42 (p18)	0.25 (p32)	0.05 (p69)

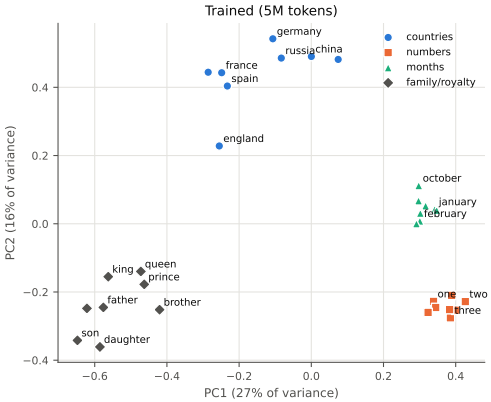
- pN: more similar than N% of random pairs among the 2,000 most frequent words
- Random frequent pairs average 0.61 (1M) and 0.32 (5M)
- Over the whole vocabulary: 0.66, because rare words stay near a shared direction
- Baseline changed to frequent words **after** the first demo evaluation; queries unchanged

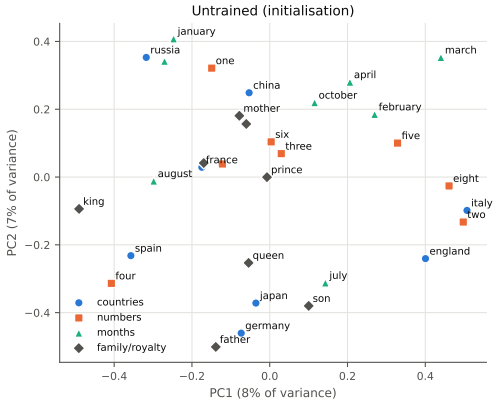
Backup: all fixed analogies

Analogy	Expected	Final: top 3	Rank of expected		
			Final	Demo	Untrained
man : *king* :: *woman* : ?	*queen*	daughter, caligula, amalric	>10	>10	>10
france : *paris* :: *italy* : ?	*rome*	interred, venice, geneva	>10	>10	>10
france : *paris* :: *germany* : ?	*berlin*	berlin, munich, exposition	1	>10	>10
man : *woman* :: *boy* : ?	*girl*	baby, girl, marple	2	>10	>10
brother : *sister* :: *father* : ?	*mother*	mother, grandfather, parents	1	>10	>10
good : *better* :: *bad* : ?	*worse*	fare, worse, cheaper	2	>10	>10
big : *bigger* :: *small* : ?	*smaller*	smaller, parking, forested	1	3	>10
england : *english* :: *france* : ?	*french*	french, spanish, portuguese	1	5	>10

Expected answer $\approx \mathbf{v}_b - \mathbf{v}_a + \mathbf{v}_c$, excluding *a*, *b*, *c*; rank among the ten best candidates.

Backup: embedding space before and after training





References

- N. Caselli (2026). *word2vec-from-scratch*. GitHub repository, `github.com/NiccoloCase/word2vec-from-scratch`.
- J. R. Firth (1957). A synopsis of linguistic theory, 1930–1955. *Studies in Linguistic Analysis*.
- Y. Goldberg, O. Levy (2014). word2vec explained. arXiv:1402.3722.
- Z. S. Harris (1954). Distributional structure. *Word* 10(2–3).
- M. Mahoney. About the test data (text8). `mattmahoney.net/dc/textdata.html`.
- T. Mikolov, K. Chen, G. Corrado, J. Dean (2013a). Efficient estimation of word representations in vector space. arXiv:1301.3781.
- T. Mikolov, I. Sutskever, K. Chen, G. Corrado, J. Dean (2013b). Distributed representations of words and phrases and their compositionality. *NeurIPS*.
- T. Mikolov, W. Yih, G. Zweig (2013c). Linguistic regularities in continuous space word representations. *NAACL-HLT*.
- J. Mu, P. Viswanath (2018). All-but-the-top: simple and effective postprocessing for word representations. *ICLR*.